

✓ Tugas Capstone Bengkel Koding

Nama : Syallom Christian

NIM : A11.2022.14384

Link file all : <https://drive.google.com/drive/folders/1b5TbkWHwxtVNNOFI72Z9qYzi8ObxFzeK?usp=sharing>

Link github : https://github.com/syallomchristian/Capstone_Project_BengKod_DataScience

Informasi Fitur Dataset

- Gender : Fitur, Kategorikal — "Jenis kelamin"
- Age : Fitur, Kontinu — "Usia"
- Height : Fitur, Kontinu — "Tinggi badan"
- Weight : Fitur, Kontinu — "Berat badan"
- family_history_with_overweight : Fitur, Biner — "Apakah ada anggota keluarga yang pernah atau sedang mengalami kelebihan berat badan?"
- FAVC : Fitur, Biner — "Apakah Anda sering mengonsumsi makanan tinggi kalori?"
- FCVC : Fitur, Integer — "Apakah Anda biasanya makan sayuran dalam setiap kali makan?"
- NCP : Fitur, Kontinu — "Berapa kali Anda makan besar dalam sehari?"
- CAEC : Fitur, Kategorikal — "Apakah Anda makan camilan di antara waktu makan?"
- SMOKE : Fitur, Biner — "Apakah Anda merokok?"
- CH2O : Fitur, Kontinu — "Berapa banyak air yang Anda minum setiap hari?"
- SCC : Fitur, Biner — "Apakah Anda memantau asupan kalori harian Anda?"
- FAF : Fitur, Kontinu — "Seberapa sering Anda melakukan aktivitas fisik?"
- TUE : Fitur, Integer — "Berapa lama Anda menggunakan perangkat teknologi seperti ponsel, video game, televisi, komputer, dan lainnya?"
- CALC : Fitur, Kategorikal — "Seberapa sering Anda mengonsumsi alkohol?"
- MTRANS : Fitur, Kategorikal — "Jenis transportasi apa yang biasa Anda gunakan?"
- NObeyesdad : Target, Kategorikal — "Tingkat obesitas"

✓ 1. Import Lib

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import math
from sklearn.preprocessing import LabelEncoder, StandardScaler
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.feature_selection import mutual_info_classif
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, conf

```

✓ 2. Import Dataset

```

from google.colab import drive
drive.mount('/content/drive', force_remount=True)

import sys
sys.path.append('/content/drive/My Drive/Project_CAPSTONE_BengKod')

df = pd.read_csv('/content/drive/My Drive/Project_CAPSTONE_BengKod/ObesityDataSet.csv')

```

 Mounted at /content/drive

```

df.info()
df.head()

# Cek distribusi kelas
print(df['NObeyesdad'].value_counts())
print("\nProporsi kelas:")
print(df['NObeyesdad'].value_counts(normalize=True))

```

 `<class 'pandas.core.frame.DataFrame'>`
 RangeIndex: 2111 entries, 0 to 2110
 Data columns (total 17 columns):

#	Column	Non-Null Count	Dtype
0	Age	2097 non-null	object
1	Gender	2102 non-null	object
2	Height	2099 non-null	object

3	weight	2100	non-null	object
4	CALC	2106	non-null	object
5	FAVC	2100	non-null	object
6	FCVC	2103	non-null	object
7	NCP	2099	non-null	object
8	SCC	2101	non-null	object
9	SMOKE	2106	non-null	object
10	CH2O	2105	non-null	object
11	family_history_with_overweight	2098	non-null	object
12	FAF	2103	non-null	object
13	TUE	2102	non-null	object
14	CAEC	2100	non-null	object
15	MTRANS	2105	non-null	object
16	NObeyesdad	2111	non-null	object

dtypes: object(17)

memory usage: 280.5+ KB

NObeyesdad

Obesity_Type_I 351

Obesity_Type_III 324

Obesity_Type_II 297

Overweight_Level_I 290

Overweight_Level_II 290

Normal_Weight 287

Insufficient_Weight 272

Name: count, dtype: int64

Proporsi kelas:

NObeyesdad

Obesity_Type_I 0.166272

Obesity_Type_III 0.153482

Obesity_Type_II 0.140692

Overweight_Level_I 0.137376

Overweight_Level_II 0.137376

Normal_Weight 0.135955

Insufficient_Weight 0.128849

Name: proportion, dtype: float64

✓ 3. EDA

47# Ringkasan informasi dataset

info = df.info()

Cek nilai yang hilang

missing_values = df.isnull().sum()

Statistik deskriptif untuk kolom numerik

desc_stats = df.describe()

info, missing_values, desc_stats

```
<class 'pandas.core.frame.DataFrame'>
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 2111 entries, 0 to 2110
```

```
Data columns (total 17 columns):
```

#	Column	Non-Null Count	Dtype
0	Age	2097 non-null	object
1	Gender	2102 non-null	object
2	Height	2099 non-null	object
3	Weight	2100 non-null	object
4	CALC	2106 non-null	object
5	FAVC	2100 non-null	object
6	FCVC	2103 non-null	object
7	NCP	2099 non-null	object
8	SCC	2101 non-null	object
9	SMOKE	2106 non-null	object
10	CH2O	2105 non-null	object
11	family_history_with_overweight	2098 non-null	object
12	FAF	2103 non-null	object
13	TUE	2102 non-null	object
14	CAEC	2100 non-null	object
15	MTRANS	2105 non-null	object
16	NObeyesdad	2111 non-null	object

```
dtypes: object(17)
```

```
memory usage: 280.5+ KB
```

```
(None,
```

Age	14
Gender	9
Height	12
Weight	11
CALC	5
FAVC	11
FCVC	8
NCP	12
SCC	10
SMOKE	5
CH2O	6
family_history_with_overweight	13
FAF	8
TUE	9
CAEC	11
MTRANS	6
NObeyesdad	0

```
dtype: int64,
```

	Age	Gender	Height	Weight	CALC	FAVC	FCVC	NCP	SCC	SMOKE	\
count	2097	2102	2099	2100	2106	2100	2103	2099	2101	2106	
unique	1394	3	1562	1518	5	3	808	637	3	3	
top	18	Male	1.7	80	Sometimes	yes	3	3	no	no	
freq	124	1056	58	58	1386	1844	647	1183	1997	2054	

	CH2O	family_history_with_overweight	FAF	TUE	CAEC	\
count	2105	2098	2103	2102	2100	
unique	1263	3	1186	1130	5	
top	2	yes	0	0	Sometimes	
freq	441	1705	404	552	1747	

```
MTRANS
```

```
NObeyesdad
```

count	2105	2111
unique	6	7

```
numerical_columns = ['Age', 'Height', 'Weight', 'FCVC', 'NCP', 'CH20', 'FAF', 'TUE']
```

```
# Ubah nilai tidak valid menjadi NaN pada kolom numerik
```

```
for col in numerical_columns:
```

```
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

```
# Hapus NaN
```

```
df_cleaned = df.dropna(subset=numerical_columns + ['NObeyesdad']) # Hapus baris dengan Na
```

```
# Buat boxplot
```

```
sns.set_theme(style="whitegrid")
```

```
for col in numerical_columns:
```

```
    plt.figure(figsize=(10, 6))
```

```
    sns.boxplot(x='NObeyesdad', y=col, hue='NObeyesdad', data=df_cleaned, palette='Set3',
```

```
    plt.title(f'Distribusi {col} berdasarkan Kategori Obesitas')
```

```
    plt.xlabel('Kategori Obesitas')
```

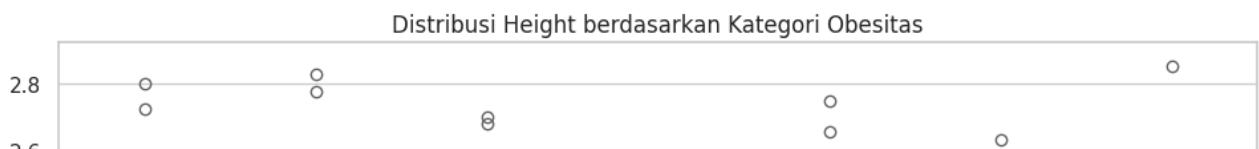
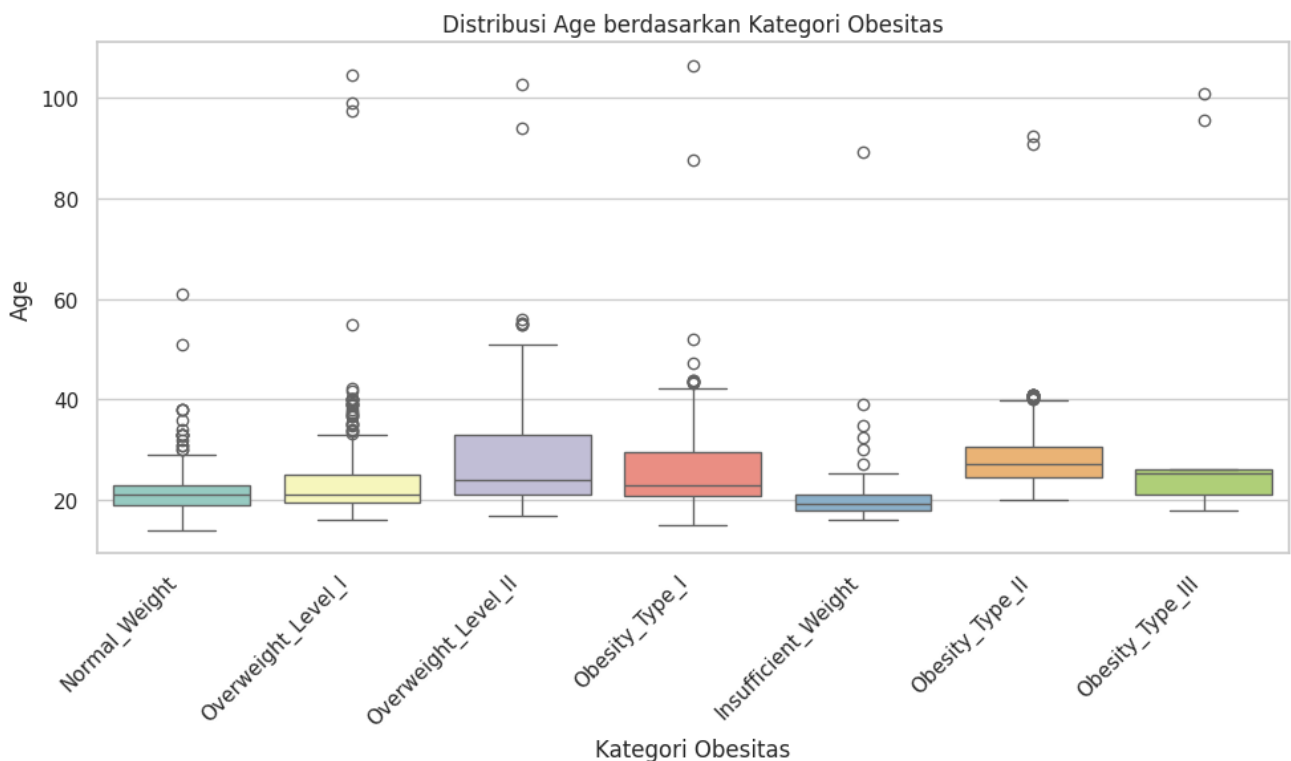
```
    plt.ylabel(col)
```

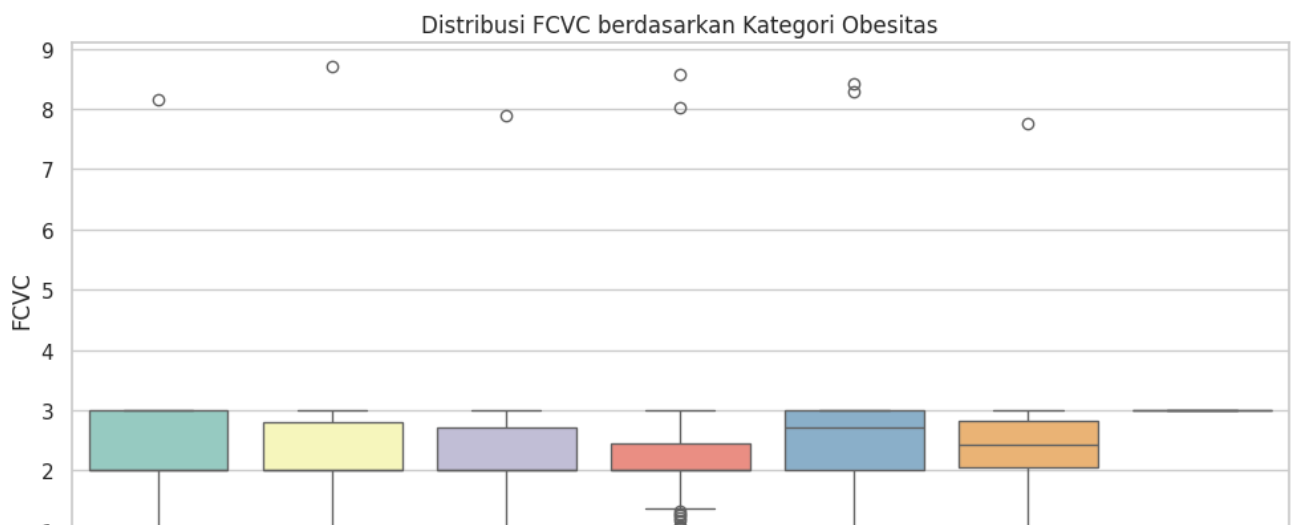
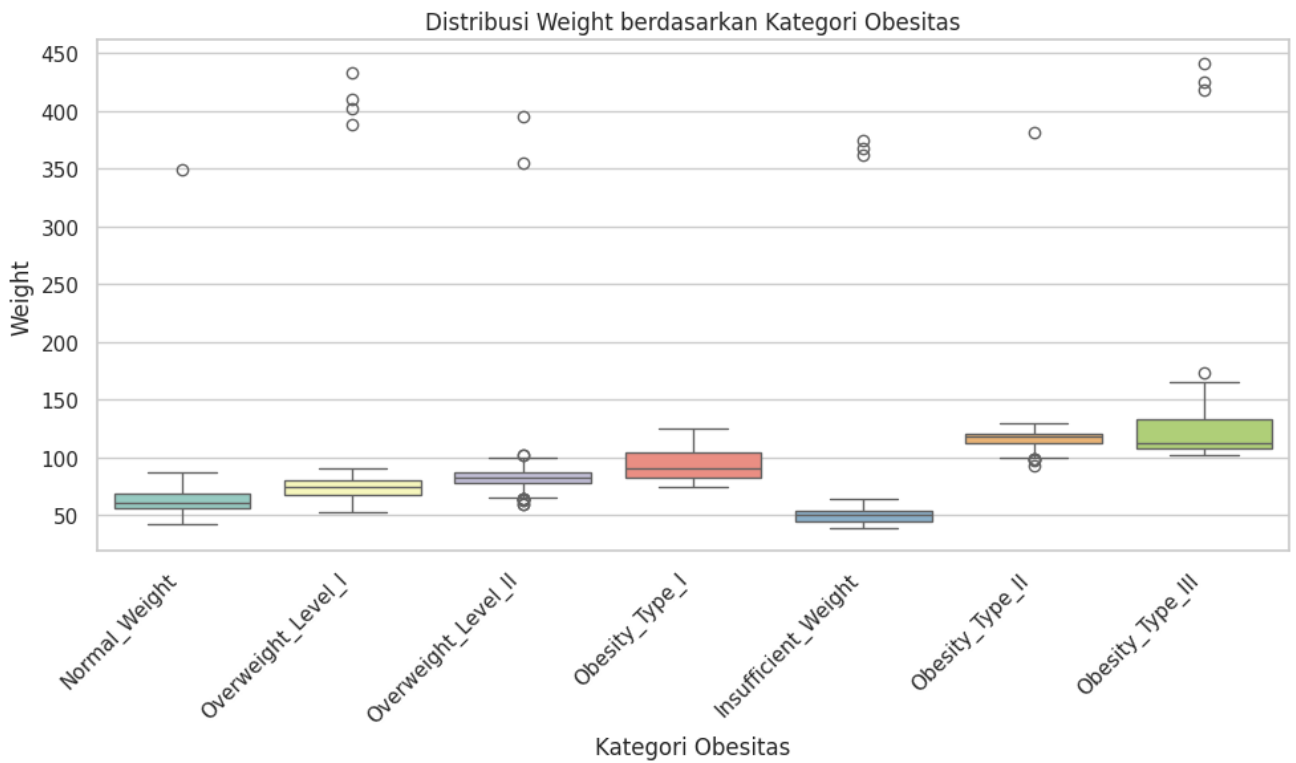
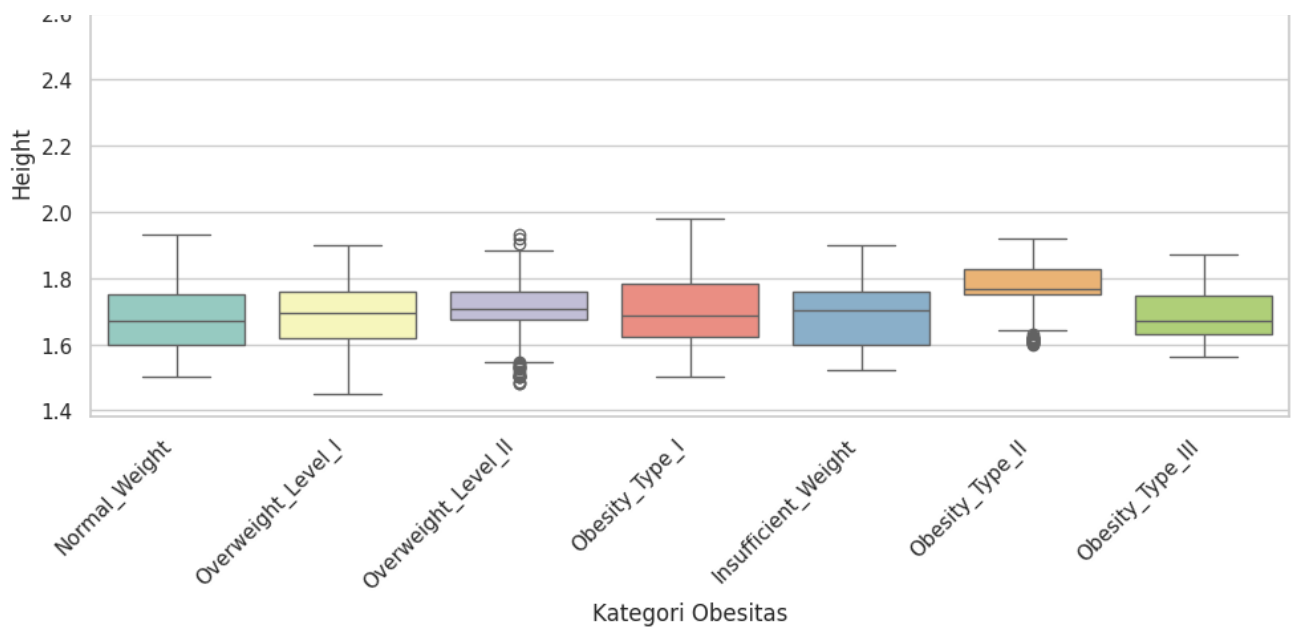
```
    plt.xticks(rotation=45, ha='right') # Putar dan rapikan label
```

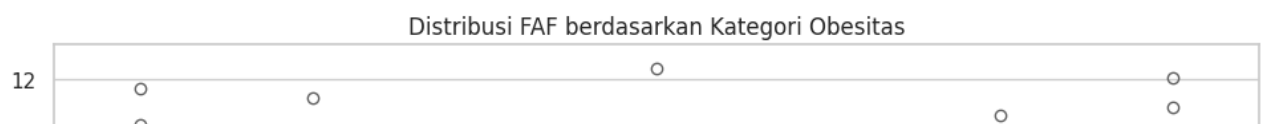
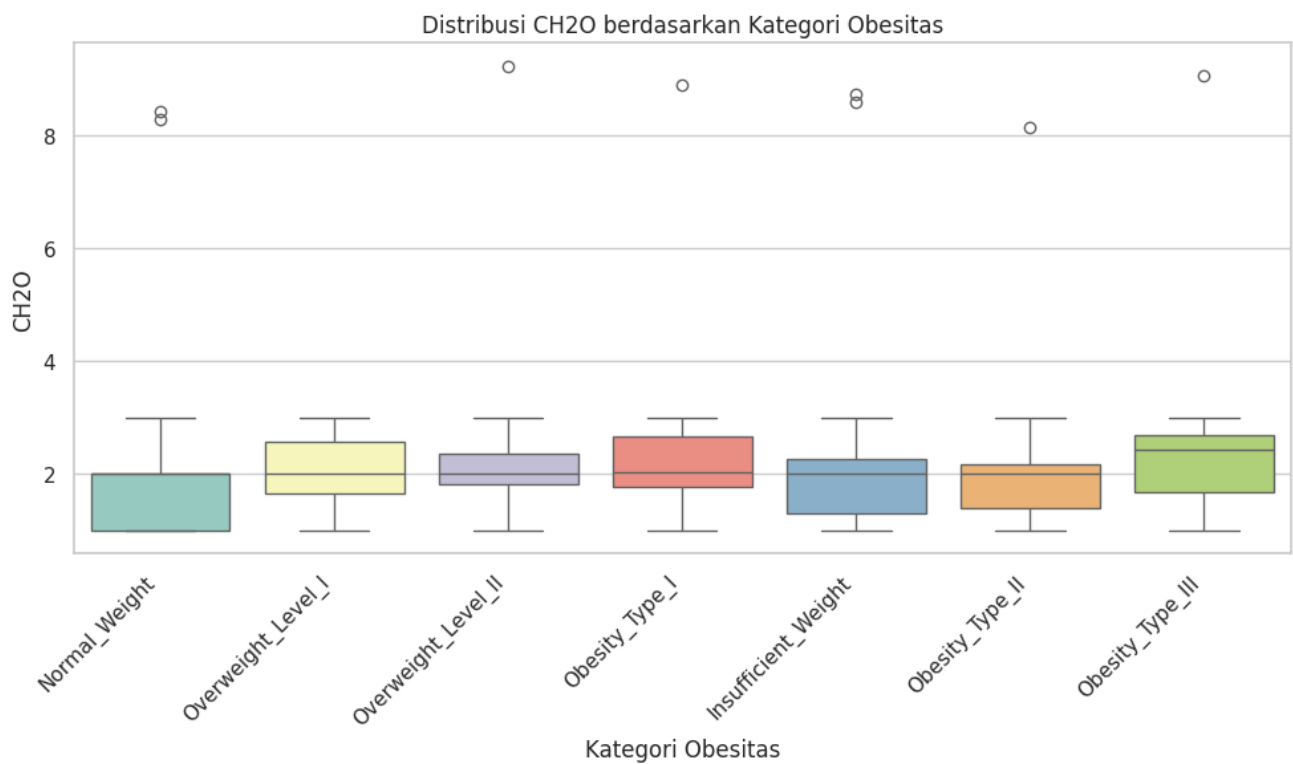
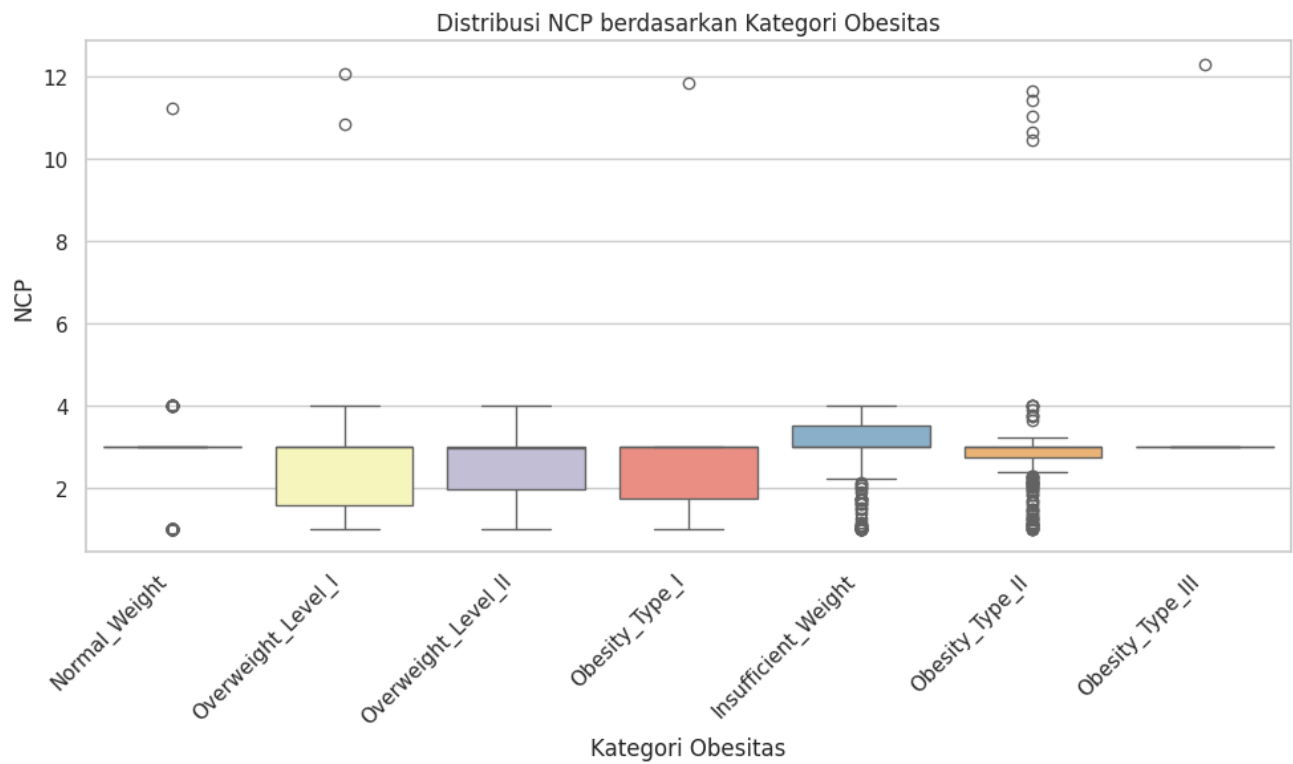
```
    plt.subplots_adjust(bottom=0.25) # Tambah jarak bawah agar tidak bertabrakan
```

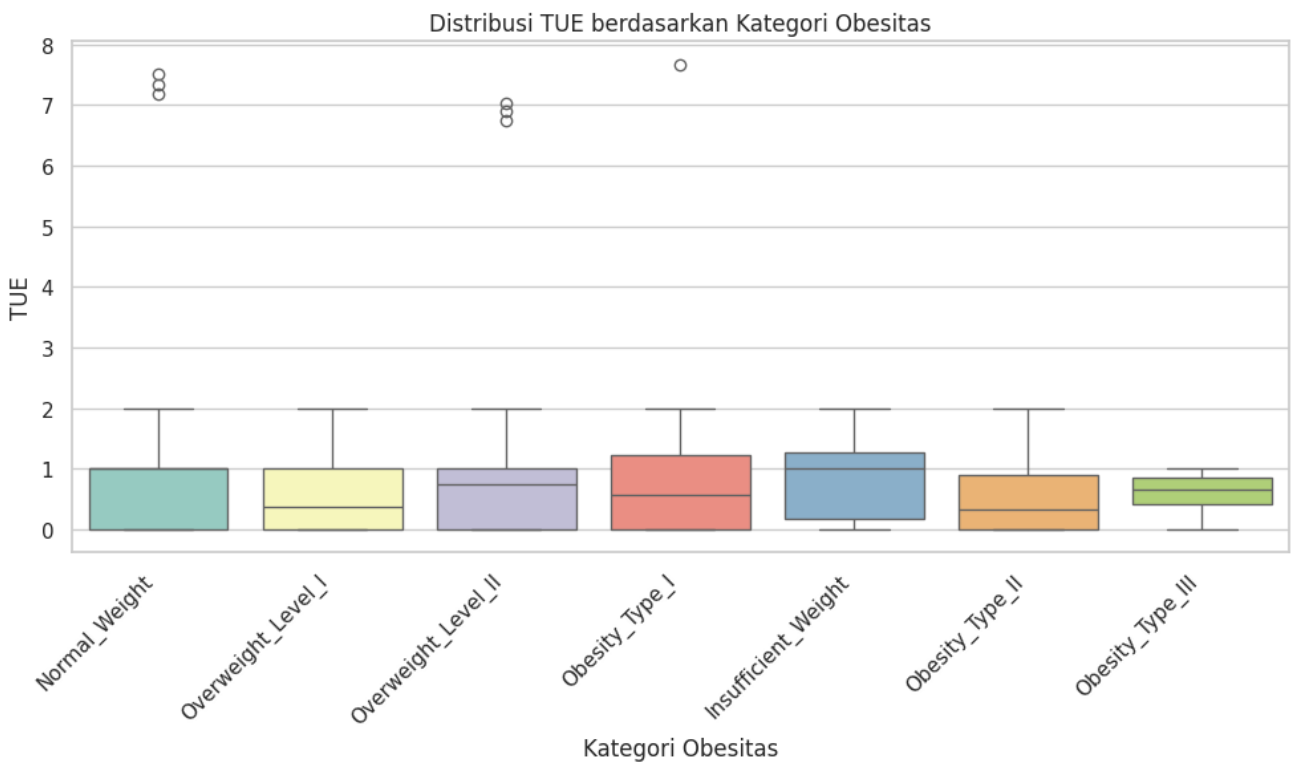
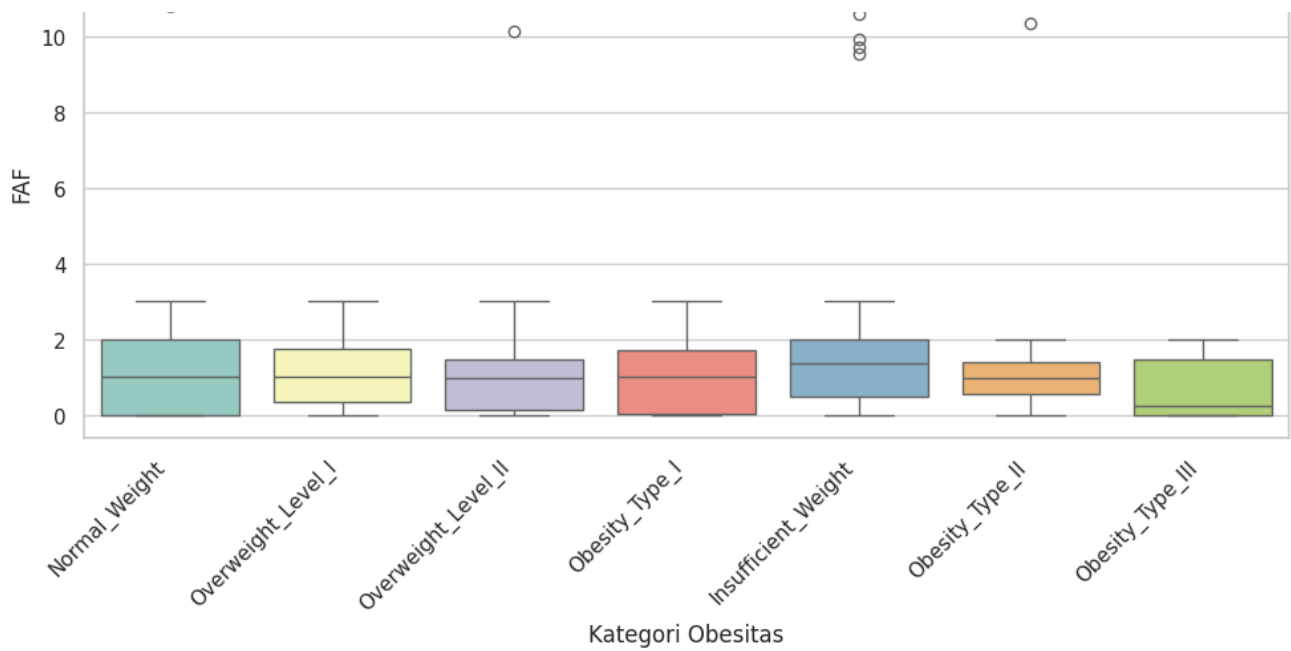
```
    plt.tight_layout()
```

```
    plt.show()
```










```
# Cek missing values per kolom
missing_values = df.isnull().sum()
#print("Jumlah Missing Values per Kolom:\n", missing_values)
print(missing_values)
```

```
Age                22
Gender              9
Height             22
Weight             19
CALC                5
FAVC               11
FCVC               18
NCP                22
SCC                10
SMOKE               5
CH20               15
family_history_with_overweight  13
FAF                19
TUE                15
CAEC               11
MTRANS             6
NObeyesdad         0
dtype: int64
```

```
# Cek unique values per kolom
unique_values = df.nunique()
print("\nJumlah Unique Values per Kolom:\n", unique_values)
```

Jumlah Unique Values per Kolom:

```

Age          1393
Gender        3
Height       1561
Weight       1517
CALC         5
FAVC         3
FCVC        807
NCP         636
SCC          3
SMOKE        3
CH2O        1262
family_history_with_overweight  3
FAF         1185
TUE         1129
CAEC         5
MTRANS       6
NObeyesdad   7
dtype: int64

```

```

print("\nUnique values pada semua kolom:")
for col in df.columns:
    print(f"- {col}: {df[col].unique()}")

```

Unique values pada semua kolom:

```

- Age: [21.      23.      27.      ... 22.524036 24.361936 23.664709]
- Gender: ['Female' 'Male' '?' nan]
- Height: [1.62      1.52      1.8      ... 1.752206 1.73945 1.738836]
- Weight: [ 64.      56.      77.      ... 133.689352 133.346641 133.472641]
- CALC: ['no' 'Sometimes' 'Frequently' '?' 'Always' nan]
- FAVC: ['no' 'yes' '?' nan]
- FCVC: [2.      3.      1.      nan 8.14899274 8.42397393
2.450218 2.880161 2.00876 2.596579 2.591439 2.392665
1.123939 2.027574 2.658112 2.88626 2.714447 2.750715
1.4925 2.205439 2.059138 2.310423 2.823179 2.052932
2.596364 2.767731 2.815157 2.737762 2.524428 2.971574
1.0816 1.270448 1.344854 2.959658 2.725282 2.844607
2.44004 2.432302 2.592247 2.449267 2.929889 2.015258
1.031149 1.592183 1.21498 1.522001 2.703436 2.362918
2.14084 2.5596 2.336044 1.813234 2.724285 2.71897
1.133844 1.757466 2.979383 2.204914 2.927218 2.88853
2.890535 2.530066 2.241606 1.003566 2.652779 2.897899
2.483979 2.945967 2.478891 2.784464 1.005578 2.938031
2.842102 1.889199 2.943749 2.33998 1.950742 2.277436
2.371338 2.984425 2.977018 2.663421 2.753752 2.318355
2.594653 2.886157 2.967853 2.619835 1.053534 2.530233
2.8813 2.824559 2.762325 2.070964 2.68601 2.794197
2.720701 2.880792 2.674431 2.55996 1.212908 1.140615
2.562409 2.004146 2.690754 2.051283 2.19005 2.21498
2.91548 2.708965 2.853513 2.580872 2.508835 2.896562
2.911877 2.910733 2.966126 2.613249 2.627031 2.919751
2.494451 1.69427 1.601236 1.204855 1.052699 2.910345
2.866383 2.913486 2.432886 2.883745 2.707666 2.919584

```

2.969205	2.486189	1.642241	1.567101	1.036414	1.649974
1.118436	2.673638	2.120185	2.34222	2.86099	2.559571
2.424977	1.786841	1.303878	1.889883	2.984004	2.749268
1.202075	8.28511134	2.341133	1.206276	2.81646	1.758394
2.577427	2.052152	2.954996	2.555401	2.108711	2.915279
1.570089	1.94313	2.903545	1.75375	2.543563	2.39728
2.37464	2.278644	1.620845	2.061952	2.838969	2.568063
2.652958	1.27785	1.729824	1.452524	2.303367	2.948425
2.291846	1.906194	1.834155	2.048582	2.948248	2.869436
2.293705	2.510583	2.366949	2.615788	2.217267	2.801514
2.188722	2.971351	2.086093	1.901611	1.977298	2.446872
2.839048	2.21232	2.427689	1.078529	1.064162	1.993101
2.620963	2.95118	2.021446	2.000466	2.5621	2.96008
2.53915	2.244142	2.253371	2.851664	1.31415	1.321028
2.253998	2.778079	2.838037	2.814453	2.013782	2.459976
2.643183	2.22399	2.104105	1.972545	2.286481	2.971588
2.872121	2.109162	2.178889	1.142468	2.047069	2.843709
2.416044	2.146598	1.766849	1.188089	1.910176	2.956671
2.002796	2.288604	2.138334	2.029634	2.048216	2.8557
2.995599	2.987148	1.887951	2.786008	2.342323	1.874935
2.213135	2.273548	2.780699	1.687569	1.989905	1.947405
2.162519	2.923916	2.99448	2.507841	1.836554	1.773265
2.388168	2.286146	2.487167	2.185938	2.206399	1.952987
2.908757	2.628791	2.749629	1.595746	2.885178	2.372494
8.7067947	2.793561	2.992329	2.927409	2.706134	2.010684
2.300408	2.119643	2.901924	2.451009	2.754646	2.417635
2.512719	1.771693	1.57223	2.661556	2.097373	2.061461
1.317729	1.882235	2.951591	2.067817	2.54527	2.694281

```
# Cek data duplikat
```

```
# Jumlah total baris duplikat (seluruh baris sama persis)
```

```
total_duplikat = df.duplicated().sum()
```

```
print("Total baris duplikat:", total_duplikat)
```

```
print("-----")
```

```
# Tampilkan baris yang terduplikat
```

```
duplikat = df[df.duplicated()]
```

```
print("Baris duplikat:")
```

```
print(duplikat)
```

```
# Ambil satu contoh baris duplikat
```

```
if not duplikat.empty:
```

```
    ref = duplikat.iloc[0]
```

```
    matching_cols = df.columns[(df == ref).all(axis=0)]
```

```
    print("Kolom yang identik di baris duplikat contoh:", list(matching_cols))
```

```
Total baris duplikat: 18
```

```
-----
```

```
Baris duplikat:
```

	Age	Gender	Height	Weight	CALC	FAVC	FCVC	NCP	SCC	SMOKE	CH2O	\
98	21.0	Female	1.52	42.0	Sometimes	no	3.0	1.0	no	no	1.0	
174	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0	

179	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
184	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
309	16.0	Female	1.66	58.0		no	2.0	1.0	no	no	1.0
460	18.0	Female	1.62	55.0		no	2.0	3.0	no	no	1.0
663	21.0	Female	1.52	42.0	Sometimes	yes	3.0	1.0	no	no	1.0
763	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
764	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
824	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
830	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
831	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
832	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
833	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
834	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
921	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
922	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0
923	21.0	Male	1.62	70.0	Sometimes	yes	2.0	1.0	no	no	3.0

	family_history_with_overweight	FAF	TUE	CAEC	\
98	no	0.0	0.0	Frequently	
174	no	1.0	0.0	no	
179	no	1.0	0.0	no	
184	no	1.0	0.0	no	
309	no	0.0	1.0	Sometimes	
460	yes	1.0	1.0	Frequently	
663	no	0.0	0.0	Frequently	
763	no	1.0	0.0	no	
764	no	1.0	0.0	no	
824	no	1.0	0.0	no	
830	no	1.0	0.0	no	
831	no	1.0	0.0	no	
832	no	1.0	0.0	no	
833	no	1.0	0.0	no	
834	no	1.0	0.0	no	
921	no	1.0	0.0	no	
922	no	1.0	0.0	no	
923	no	1.0	0.0	no	

	MTRANS	NObeyesdad
98	Public_Transportation	Insufficient_Weight
174	Public_Transportation	Overweight_Level_I
179	Public_Transportation	Overweight_Level_I
184	Public_Transportation	Overweight_Level_I
309	Walking	Normal_Weight
460	Public_Transportation	Normal_Weight
663	Public_Transportation	Insufficient_Weight
763	Public_Transportation	Overweight_Level_I
764	Public_Transportation	Overweight_Level_I
824	Public_Transportation	Overweight_Level_I
830	Public_Transportation	Overweight_Level_I
831	Public_Transportation	Overweight_Level_I
832	Public_Transportation	Overweight_Level_I
833	Public_Transportation	Overweight_Level_I

```
# Cek keseimbangan data pada kolom target 'NObeyesdad'
class distribution = df['NObeyesdad'].value counts()
```

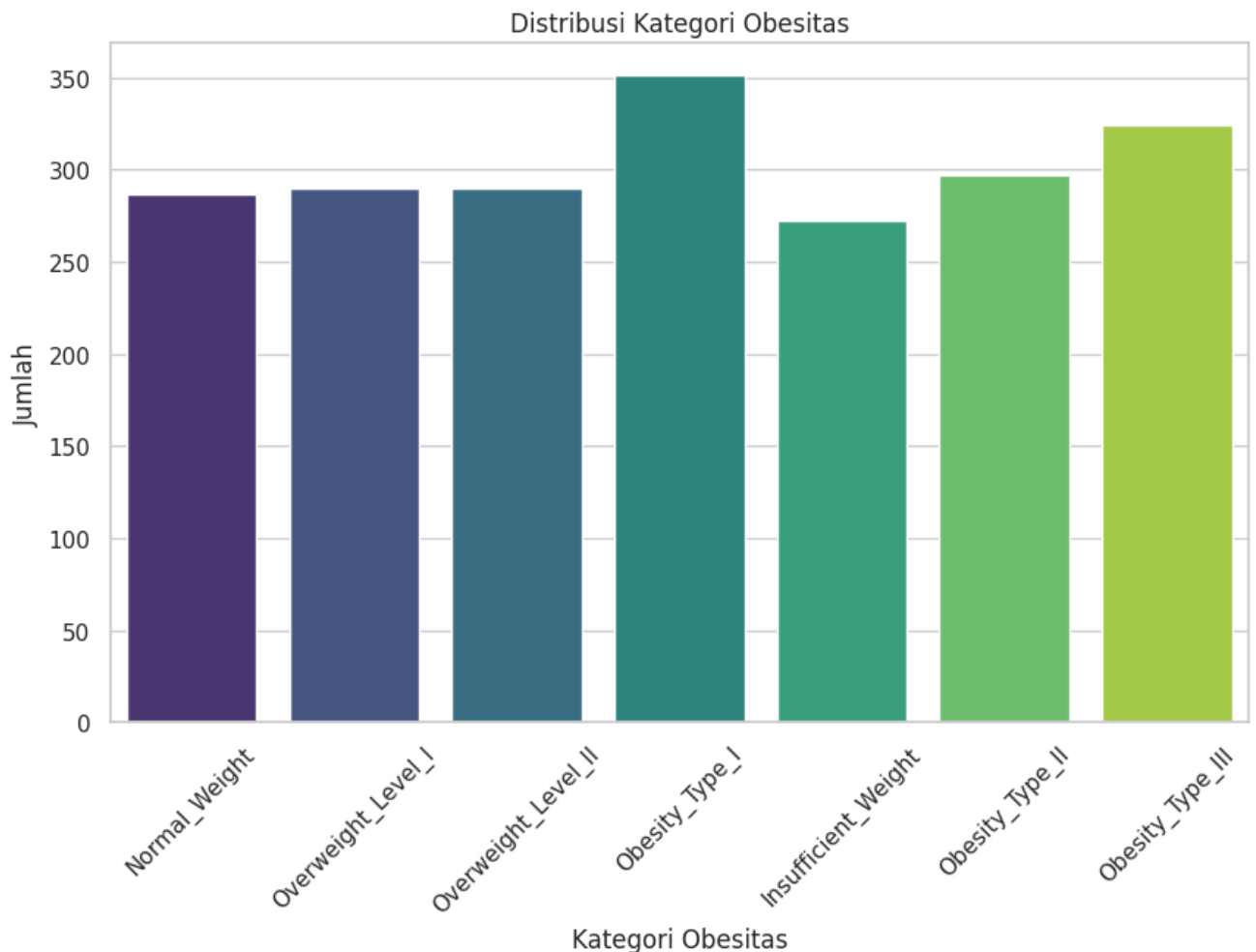
```
print("\nDistribusi Keseimbangan Data (NObeyesdad):\n", class_distribution)
```

```
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='NObeyesdad', hue='NObeyesdad', palette='viridis', legend=False)
plt.title('Distribusi Kategori Obesitas')
plt.xlabel('Kategori Obesitas')
plt.ylabel('Jumlah')
plt.xticks(rotation=45)
plt.show()
```

Distribusi Keseimbangan Data (NObeyesdad):

NObeyesdad	
Obesity_Type_I	351
Obesity_Type_III	324
Obesity_Type_II	297
Overweight_Level_I	290
Overweight_Level_II	290
Normal_Weight	287
Insufficient_Weight	272

Name: count, dtype: int64



```

# Konversi kolom numerik ke tipe data numerik
for col in numerical_columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Hapus baris dengan missing values di kolom numerik
df_cleaned = df.dropna(subset=numerical_columns)

# Hitung matriks korelasi
correlation_matrix = df_cleaned[numerical_columns].corr()

# Ambil nilai korelasi absolut dan urutkan
abs_corr_matrix = np.abs(correlation_matrix)
upper_triangle = abs_corr_matrix.where(np.triu(np.ones(abs_corr_matrix.shape), k=1).astype(bool))
strong_correlations = upper_triangle.unstack().sort_values(ascending=False)
strong_correlations = strong_correlations.dropna() # Hapus NaN

# Pilih 7 korelasi teratas
top_7_correlations = strong_correlations.head(7)

# Siapkan data untuk diagram batang
correlation_data = pd.DataFrame({
    'Pair': [f"{pair[0]} - {pair[1]}" for pair in top_7_correlations.index],
    'Correlation': top_7_correlations.values
})

# Visualisasikan dalam diagram batang
plt.figure(figsize=(10, 6))
sns.barplot(x='Pair', y='Correlation', data=correlation_data, palette='viridis')
plt.title('7 Korelasi Terkuat Antar Kolom Numerik')
plt.xlabel('Pasangan Kolom')
plt.ylabel('Korelasi Absolut')
plt.xticks(rotation=45, ha='right') # Rotasi label sumbu x agar terbaca
plt.tight_layout() # Untuk mencegah label tumpang tindih
plt.show()

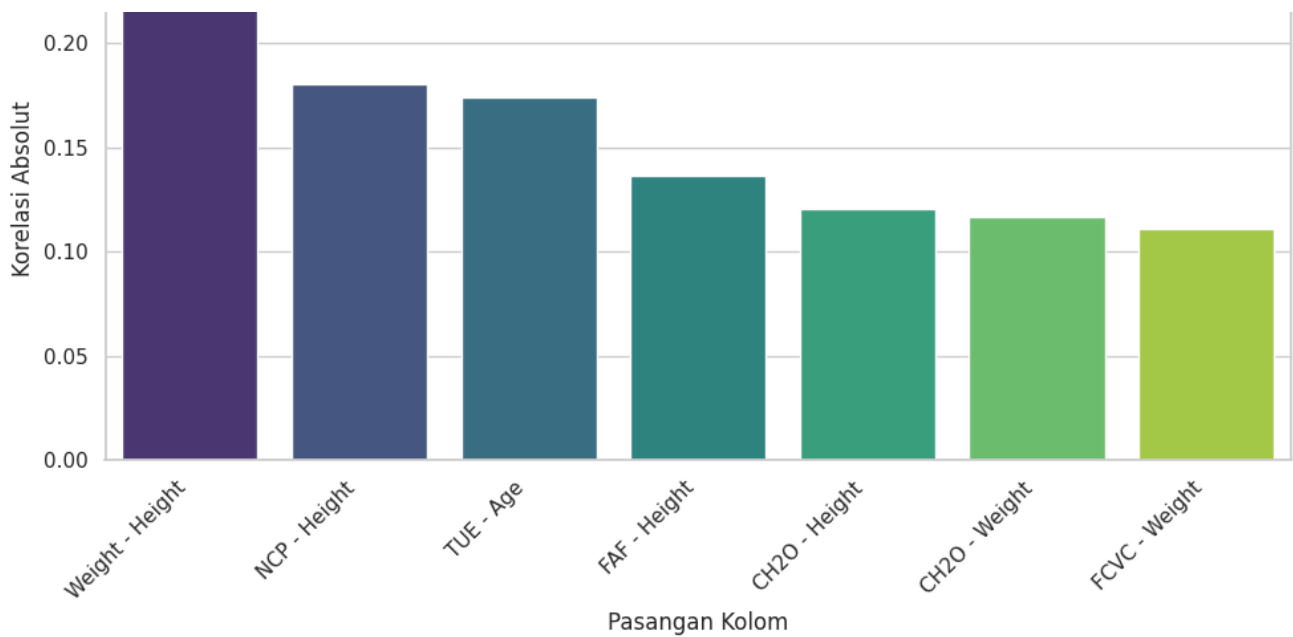
```

<ipython-input-11-2380615929>:28: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.

```
sns.barplot(x='Pair', y='Correlation', data=correlation_data, palette='viridis')
```





Top 7 Korelasi Tertinggi (dalam nilai absolut)

Pasangan Fitur	Korelasi	Analisis
Weight – Height	0.248	Positif lemah: Orang dengan tinggi badan lebih besar cenderung memiliki berat lebih tinggi, meskipun
NCP – Height	0.180	Korelasi lemah: Orang dengan tinggi tertentu cenderung memiliki pola makan besar tertentu, tetapi
TUE – Age	0.174	Korelasi lemah: Usia memengaruhi durasi penggunaan teknologi; kemungkinan, kelompok usia muc
FAF – Height	0.136	Korelasi sangat lemah: Tinggi badan sedikit berkorelasi dengan aktivitas fisik, bisa jadi orang lebih t
CH2O – Height	0.121	Korelasi sangat lemah: Tinggi badan sedikit berkaitan dengan konsumsi air harian, tetapi tidak sign
CH2O – Weight	0.117	Korelasi sangat lemah: Berat badan memiliki sedikit hubungan dengan konsumsi air, tetapi tidak cu
FCVC – Weight	0.111	Korelasi sangat lemah: Konsumsi sayuran berkaitan sedikit dengan berat badan; bisa berarti diet se

Kesimpulan Analisis Korelasi

- Tidak ada korelasi yang **kuat** antar fitur numerik (semuanya < 0.3).
- Korelasi tertinggi pun (Weight – Height) hanya 0.25, yang termasuk **lemah**.
- Ini menunjukkan bahwa **tidak ada dua fitur numerik** dalam dataset ini yang sangat linear satu sama lain.

- Hal ini baik untuk pemodelan, karena tidak ada multikolinearitas tinggi yang bisa merusak performa model prediktif berbasis regresi atau pohon keputusan.

✓ 4. Preprocessing Data

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2111 entries, 0 to 2110
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   2089 non-null   float64
1   Gender                               2102 non-null   object
2   Height                               2089 non-null   float64
3   Weight                               2092 non-null   float64
4   CALC                                 2106 non-null   object
5   FAVC                                 2100 non-null   object
6   FCVC                                 2093 non-null   float64
7   NCP                                  2089 non-null   float64
8   SCC                                  2101 non-null   object
9   SMOKE                                2106 non-null   object
10  CH2O                                 2096 non-null   float64
11  family_history_with_overweight       2098 non-null   object
12  FAF                                   2092 non-null   float64
13  TUE                                  2096 non-null   float64
14  CAEC                                 2100 non-null   object
15  MTRANS                               2105 non-null   object
16  NObeyesdad                           2111 non-null   object
dtypes: float64(8), object(9)
memory usage: 280.5+ KB
```

```
le_gender = LabelEncoder()
```

```
le_family = LabelEncoder()
```

```
df['Gender'] = le_gender.fit_transform(df['Gender']).astype(float)
```

```
df['family_history_with_overweight'] = le_family.fit_transform(df['family_history_with_ov
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2111 entries, 0 to 2110
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   2089 non-null   float64
1   Gender                               2111 non-null   float64
2   Height                               2089 non-null   float64
```

```

3   Weight                2092 non-null    float64
4   CALC                  2106 non-null    object
5   FAVC                  2100 non-null    object
6   FCVC                  2093 non-null    float64
7   NCP                   2089 non-null    float64
8   SCC                   2101 non-null    object
9   SMOKE                 2106 non-null    object
10  CH20                  2096 non-null    float64
11  family_history_with_overweight  2111 non-null    float64
12  FAF                   2092 non-null    float64
13  TUE                   2096 non-null    float64
14  CAEC                  2100 non-null    object
15  MTRANS                2105 non-null    object
16  NObeyesdad            2111 non-null    object

```

```
dtypes: float64(10), object(7)
```

```
memory usage: 280.5+ KB
```

```
# Kolom-kolom numerik yang perlu dikonversi
```

```
numerical_columns = ['Age', 'Height', 'Weight', 'FCVC', 'NCP', 'CH20', 'FAF', 'TUE']
```

```
numerical_columns = ['Age', 'Gender', 'family_history_with_overweight', 'Height', 'Weight']
```

```
# Loop melalui kolom numerik dan coba konversi ke numeric, errors='coerce' akan mengubah
for col in numerical_columns:
```

```
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 2111 entries, 0 to 2110
```

```
Data columns (total 17 columns):
```

#	Column	Non-Null Count	Dtype
0	Age	2089 non-null	float64
1	Gender	2111 non-null	float64
2	Height	2089 non-null	float64
3	Weight	2092 non-null	float64
4	CALC	2106 non-null	object
5	FAVC	2100 non-null	object
6	FCVC	2093 non-null	float64
7	NCP	2089 non-null	float64
8	SCC	2101 non-null	object
9	SMOKE	2106 non-null	object
10	CH20	2096 non-null	float64
11	family_history_with_overweight	2111 non-null	float64
12	FAF	2092 non-null	float64
13	TUE	2096 non-null	float64
14	CAEC	2100 non-null	object
15	MTRANS	2105 non-null	object
16	NObeyesdad	2111 non-null	object

```
dtypes: float64(10), object(7)
```

```
memory usage: 280.5+ KB
```

```
# Setelah konversi, Anda mungkin memiliki missing values (NaN) yang perlu ditangani
# Anda bisa memilih untuk menghapusnya atau melakukan imputasi
df_cleaned = df.dropna(subset=numerical_columns + ['NObeyesdad']) # Hapus baris dengan Na
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2111 entries, 0 to 2110
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   2089 non-null   float64
1   Gender                               2111 non-null   float64
2   Height                               2089 non-null   float64
3   Weight                               2092 non-null   float64
4   CALC                                 2106 non-null   object
5   FAVC                                 2100 non-null   object
6   FCVC                                 2093 non-null   float64
7   NCP                                  2089 non-null   float64
8   SCC                                  2101 non-null   object
9   SMOKE                                2106 non-null   object
10  CH20                                 2096 non-null   float64
11  family_history_with_overweight       2111 non-null   float64
12  FAF                                   2092 non-null   float64
13  TUE                                  2096 non-null   float64
14  CAEC                                 2100 non-null   object
15  MTRANS                               2105 non-null   object
16  NObeyesdad                           2111 non-null   object
dtypes: float64(10), object(7)
memory usage: 280.5+ KB
```

```
# Tentukan kolom numerik berdasarkan tipe data atau nama kolom yang sesuai
numerical_columns = ['Age', 'Height', 'Weight', 'FCVC', 'NCP', 'CH20', 'FAF', 'TUE']
numerical_columns = ['Age', 'Gender', 'family_history_with_overweight', 'Height', 'Weight']
```

```
# Pastikan kolom numerik bertipe numerik
df[numerical_columns] = df[numerical_columns].apply(pd.to_numeric, errors='coerce')
```

```
# 1. Tangani missing values (diisi dengan median untuk numerik dan modus untuk kategorika
```

```
# Isi NaN pada kolom numerik secara implisit dengan median masing-masing
df[numerical_columns] = df[numerical_columns].fillna(df[numerical_columns].median())
```

```
# Tentukan kolom kategorikal selain kolom numerik
categorical_columns = [col for col in df.columns if col not in numerical_columns]
```

```
# Isi NaN pada kolom kategorikal dengan modus masing-masing
for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2111 entries, 0 to 2110
Data columns (total 17 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Age                                  2111 non-null   float64
 1   Gender                              2111 non-null   float64
 2   Height                             2111 non-null   float64
 3   Weight                             2111 non-null   float64
 4   CALC                               2111 non-null   object
 5   FAVC                               2111 non-null   object
 6   FCVC                               2111 non-null   float64
 7   NCP                                2111 non-null   float64
 8   SCC                                2111 non-null   object
 9   SMOKE                              2111 non-null   object
10  CH2O                               2111 non-null   float64
11  family_history_with_overweight     2111 non-null   float64
12  FAF                                 2111 non-null   float64
13  TUE                                2111 non-null   float64
14  CAEC                               2111 non-null   object
15  MTRANS                             2111 non-null   object
16  NObeyesdad                         2111 non-null   object
dtypes: float64(10), object(7)
memory usage: 280.5+ KB

```

```

# cek data awal, dupe, hapus dupe
print("Data awal:", len(df))
print("Data duplikat:", df.duplicated().sum())
df.drop_duplicates(inplace=True)
print("Data setelah duplikat dihapus:", len(df))

```

```

Data awal: 2111
Data duplikat: 19
Data setelah duplikat dihapus: 2092

```

```

# Show outlier menggunakan boxplot
# import matplotlib.pyplot as plt
# import seaborn as sns
# plt.figure(figsize=(15, 8))
# for i, col in enumerate(numerical_columns, 1):
#     plt.subplot(2, 4, i)
#     sns.boxplot(data=df, x=col)
#     plt.title(f'Boxplot - {col}')
# plt.tight_layout()
# plt.show()
n_cols = 4 # jumlah kolom subplot per baris
n = len(numerical_columns)
n_rows = math.ceil(n / n_cols)

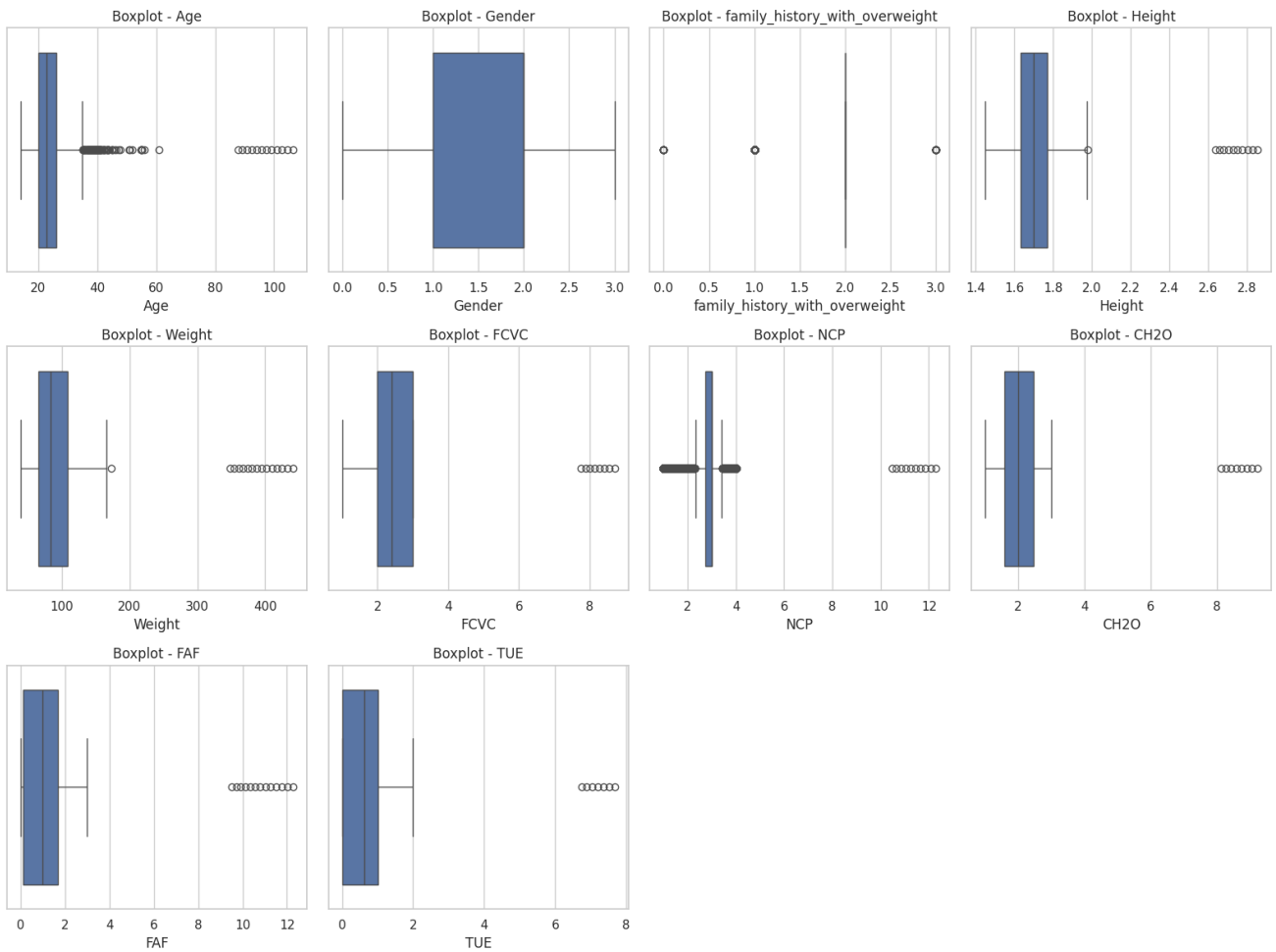
plt.figure(figsize=(n_cols * 4, n_rows * 4))
for i, col in enumerate(numerical_columns, 1):
    plt.subplot(n_rows, n_cols, i)

```

```

plt.subplot(n_rows, n_cols, i)
sns.boxplot(data=df, x=col)
plt.title(f'Boxplot - {col}')
plt.tight_layout()
plt.show()

```



```
# Cek distribusi class NObeyesdad sebelum SMOTE
y = df['NObeyesdad']
print("Distribusi class sebelum SMOTE:")
print(y.value_counts())
```

```
Distribusi class sebelum SMOTE:
NObeyesdad
Obesity_Type_I      351
Obesity_Type_III    324
Obesity_Type_II     297
Overweight_Level_II 290
Normal_Weight       285
Overweight_Level_I  276
Insufficient_Weight 269
Name: count, dtype: int64
```

```
# Fix ketidakseimbangan kelas data dengan oversampling menggunakan SMOTE
```

```
# Pisah fitur target
X = df[numerical_columns + categorical_columns[:-1]] # kecuali kolom target
y = df['NObeyesdad']
```

```
# Fitur kategorikal perlu diubah ke numerik
# pakai one-hot encoding
X_encoded = pd.get_dummies(X, drop_first=True)
```

```
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_encoded, y)
```

```
print("Distribusi kelas setelah SMOTE:")
print(pd.Series(y_resampled).value_counts())
```

```
Distribusi kelas setelah SMOTE:
NObeyesdad
Normal_Weight      351
Overweight_Level_I 351
Overweight_Level_II 351
Obesity_Type_I     351
Insufficient_Weight 351
Obesity_Type_II    351
Obesity_Type_III   351
Name: count, dtype: int64
```

```
# Cek apakah sudah normalisasi dan standarisasi pada kolom numerik
print("Statistik kolom numerik di X_encoded:")
print(X_encoded[numerical_columns].describe())
```

```
# Jika mean ~0 dan std ~1, kemungkinan sudah distandarisasi.
# Jika min ~0 dan max ~1, kemungkinan sudah dinormalisasi.
```

```
Statistik kolom numerik di X_encoded:
```

	Age	Gender	family_history_with_overweight	Height \
count	2092.000000	2092.000000	2092.000000	2092.000000
mean	24.736938	1.500000	1.821702	1.707382
std	8.376500	0.520732	0.410581	0.117238
min	14.000000	0.000000	0.000000	1.450000
25%	19.989818	1.000000	2.000000	1.631633
50%	22.814657	2.000000	2.000000	1.701284
75%	26.000000	2.000000	2.000000	1.770000
max	106.441252	3.000000	3.000000	2.853986

	Weight	FCVC	NCP	CH2O	FAF \
count	2092.000000	2092.000000	2092.000000	2092.000000	2092.000000
mean	88.774054	2.446193	2.746059	2.025838	1.075347
std	36.075253	0.641441	0.968158	0.732983	1.148408
min	39.000000	1.000000	1.000000	1.000000	0.000000
25%	66.000000	2.000000	2.723920	1.587615	0.131132
50%	83.000000	2.397284	3.000000	2.000000	1.000000
75%	108.055736	3.000000	3.000000	2.458433	1.683612
max	441.131078	8.706795	12.299828	9.233294	12.270275

	TUE
count	2092.000000
mean	0.684011
std	0.714729
min	0.000000
25%	0.000000
50%	0.625360
75%	1.000000
max	7.669504

```

scaler = StandardScaler()
X_resampled_scaled = X_resampled.copy()
X_resampled_scaled[numerical_columns] = scaler.fit_transform(X_resampled[numerical_columns])

print("Statistik setelah standarisasi:")
print(X_resampled_scaled[numerical_columns].describe())

```

Statistik setelah standarisasi:

	Age	Gender	family_history_with_overweight	Height \
count	2.457000e+03	2.457000e+03	2.457000e+03	
mean	-1.792985e-16	1.012169e-16	2.544882e-16	
std	1.000204e+00	1.000204e+00	1.000204e+00	
min	-1.276413e+00	-2.908855e+00	-4.375225e+00	
25%	-5.715689e-01	-9.738907e-01	4.586170e-01	
50%	-2.208028e-01	8.168209e-01	4.586170e-01	
75%	1.606618e-01	9.610737e-01	4.586170e-01	
max	9.794000e+00	2.896038e+00	2.875538e+00	

	Weight	FCVC	NCP	CH2O	\
count	2.457000e+03	2.457000e+03	2.457000e+03	2.457000e+03	
mean	-1.792985e-16	-4.164353e-16	-9.254118e-17	-1.735147e-17	-4.222191e-16
std	1.000204e+00	1.000204e+00	1.000204e+00	1.000204e+00	1.000204e+00
min	-2.165053e+00	-1.365290e+00	-2.319118e+00	-1.811701e+00	-1.419083e+00
25%	-6.320535e-01	-6.309735e-01	-7.089946e-01	-6.403428e-02	-5.832605e-01

50%	-4.377101e-02	-1.410174e-01	-6.931808e-02	2.530141e-01	-2.868745e-02
75%	5.047269e-01	5.305758e-01	9.011294e-01	2.530141e-01	5.778878e-01
max	9.597297e+00	1.000538e+01	1.008978e+01	9.853762e+00	1.002845e+01

	FAF	TUE
count	2.457000e+03	2.457000e+03
mean	-1.156765e-17	1.156765e-17
std	1.000204e+00	1.000204e+00
min	-9.754452e-01	-9.910137e-01
25%	-8.100115e-01	-9.850457e-01
50%	-8.080746e-02	-6.503931e-02
75%	5.447062e-01	4.437158e-01
max	1.000201e+01	1.001265e+01

```
# Cek apakah sudah normalisasi dan standarisasi pada kolom numerik
print("Statistik kolom numerik di X_encoded:")
print(X_encoded[numerical_columns].describe())
```

```
# Jika mean ~0 dan std ~1, kemungkinan sudah distandarisasi.
# Jika min ~0 dan max ~1, kemungkinan sudah dinormalisasi.
```

Statistik kolom numerik di X_encoded:

	Age	Gender	family_history_with_overweight	Height \
count	2092.000000	2092.000000	2092.000000	2092.000000
mean	24.736938	1.500000	1.821702	1.707382
std	8.376500	0.520732	0.410581	0.117238
min	14.000000	0.000000	0.000000	1.450000
25%	19.989818	1.000000	2.000000	1.631633
50%	22.814657	2.000000	2.000000	1.701284
75%	26.000000	2.000000	2.000000	1.770000
max	106.441252	3.000000	3.000000	2.853986

	Weight	FCVC	NCP	CH2O	FAF \
count	2092.000000	2092.000000	2092.000000	2092.000000	2092.000000
mean	88.774054	2.446193	2.746059	2.025838	1.075347
std	36.075253	0.641441	0.968158	0.732983	1.148408
min	39.000000	1.000000	1.000000	1.000000	0.000000
25%	66.000000	2.000000	2.723920	1.587615	0.131132
50%	83.000000	2.397284	3.000000	2.000000	1.000000
75%	108.055736	3.000000	3.000000	2.458433	1.683612
max	441.131078	8.706795	12.299828	9.233294	12.270275

	TUE
count	2092.000000
mean	0.684011
std	0.714729
min	0.000000
25%	0.000000
50%	0.625360
75%	1.000000
max	7.669504

```
# Simpan dataset bersih hasil preprocessing ke file CSV
```



```
df.to_csv('ObesityDataSet_Final.csv', index=False)
```

5. Pemodelan

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2092 entries, 0 to 2110
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   2092 non-null   float64
1   Gender                               2092 non-null   float64
2   Height                               2092 non-null   float64
3   Weight                               2092 non-null   float64
4   CALC                                 2092 non-null   object
5   FAVC                                 2092 non-null   object
6   FCVC                                 2092 non-null   float64
7   NCP                                  2092 non-null   float64
8   SCC                                  2092 non-null   object
9   SMOKE                                2092 non-null   object
10  CH2O                                  2092 non-null   float64
11  family_history_with_overweight       2092 non-null   float64
12  FAF                                   2092 non-null   float64
13  TUE                                   2092 non-null   float64
14  CAEC                                 2092 non-null   object
15  MTRANS                               2092 non-null   object
16  NObeyesdad                           2092 non-null   object
dtypes: float64(10), object(7)
memory usage: 294.2+ KB
```

```
#pemilihan fitur dan target
```

```
features = ["Age", "Height", "Weight", "Gender", "family_history_with_overweight"]
```

```
# ", "
```

```
x = df[features]
```

```
y = df['NObeyesdad']
```

```
scaler = StandardScaler()
```

```
x = scaler.fit_transform(x)
```

```
#split data
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
#Evaluasi performa model
```

```
#from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, cla
```

```
def evaluate_model(model, x_train, x_test, y_train, y_test):
```

```
    model.fit(x_train, y_train)
```

```
y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
report = classification_report(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

return accuracy, precision, recall, f1, report, conf_matrix

#Memakai model Decision Tree dan Random Forest
models = {
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
}

#Menampilkan hasil Model
result = {}
for name, model in models.items():
    accuracy, precision, recall, f1, report, conf_matrix = evaluate_model(model, x_train,

    result[name] = {
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1 Score": f1,
        "Classification Report": report,
        "Confusion Matrix": conf_matrix
    }

for name, result in result.items():
    print(f"Model: {name}")
    print(f"Accuracy: {result['Accuracy']:.4f}")
    print(f"Precision: {result['Precision']:.4f}")
    print(f"Recall: {result['Recall']:.4f}")
    print(f"F1 Score: {result['F1 Score']:.4f}")
    print("Classification Report:\n", result['Classification Report'])
    print("Confusion Matrix:\n", result['Confusion Matrix'])
    print("-" * 50)

    Model: Decision Tree
    Accuracy: 0.9117
    Precision: 0.9139
    Recall: 0.9117
```

```
recall: 0.9117
```

```
F1 Score: 0.9118
```

```
Classification Report:
```

	precision	recall	f1-score	support
Insufficient_Weight	0.96	0.93	0.95	58
Normal_Weight	0.85	0.91	0.88	55
Obesity_Type_I	0.95	0.91	0.93	66
Obesity_Type_II	0.90	0.97	0.93	64
Obesity_Type_III	0.94	0.95	0.95	65
Overweight_Level_I	0.82	0.86	0.84	49
Overweight_Level_II	0.95	0.84	0.89	62
accuracy			0.91	419
macro avg	0.91	0.91	0.91	419
weighted avg	0.91	0.91	0.91	419

```
Confusion Matrix:
```

```
[[54  3  0  1  0  0  0]
 [ 0 50  0  0  0  5  0]
 [ 0  1 60  3  1  0  1]
 [ 0  0  1 62  1  0  0]
 [ 2  0  1  0 62  0  0]
 [ 0  3  0  2  0 42  2]
 [ 0  2  1  1  2  4 52]]
```

```
-----
Model: Random Forest
```

```
Accuracy: 0.9284
```

```
Precision: 0.9292
```

```
Recall: 0.9284
```

```
F1 Score: 0.9283
```

```
Classification Report:
```

	precision	recall	f1-score	support
Insufficient_Weight	0.96	0.90	0.93	58
Normal_Weight	0.86	0.93	0.89	55
Obesity_Type_I	0.95	0.92	0.94	66
Obesity_Type_II	0.95	0.98	0.97	64
Obesity_Type_III	0.96	0.98	0.97	65
Overweight_Level_I	0.88	0.88	0.88	49
Overweight_Level_II	0.92	0.89	0.90	62
accuracy			0.93	419
macro avg	0.93	0.93	0.93	419
weighted avg	0.93	0.93	0.93	419

```
Confusion Matrix:
```

```
[[52  5  0  1  0  0  0]
 [ 2 51  0  0  0  1  1]
 [ 0  1 61  1  0  2  1]
 [ 0  0  0 63  1  0  0]
 [ 0  0  0  0 64  1  0]
 [ 0  1  0  0  2 43  3]
 [ 0  1  3  1  0  2 55]]
```

